

Paimon Iceberg 模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-iceberg

版本分支：release-1.3.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月28日

第1章 模块概述

1.1 模块定位

paimon-iceberg 模块是 Apache Paimon 项目中负责 Iceberg REST Catalog 元数据兼容性的核心扩展模块。该模块通过实现 IcebergMetadataCommitter 接口，将 Paimon 表的元数据以 Iceberg 标准格式提交到 Iceberg REST Catalog 服务中，从而使 Iceberg 读取引擎（如 Spark、Flink、Trino 等）能够直接读取 Paimon 管理的原始数据文件。

该模块是 Paimon Iceberg 兼容层中的一个可插拔组件，通过 Java SPI (Service Provider Interface) 机制被自动发现和加载。在 paimon-core 模块中，IcebergCommitCallback 作为提交回调在每次快照提交后被调用，它会通过 SPI 发现并使用 paimon-iceberg 模块中注册的 IcebergRESTMetadataCommitterFactory 来创建 IcebergRestMetadataCommitter 实例。

1.2 核心目标

- REST Catalog 集成：将 Paimon 快照提交后的 Iceberg 元数据同步到 Iceberg REST Catalog 服务
- Schema 兼容性处理：解决 Paimon (field ID 从 0 开始) 与 Iceberg (field ID 从 1 开始) 之间的 Schema ID 偏移问题
- 增量/全量提交：支持基于 base metadata 的增量更新和无 base 的全量重建两种模式
- 分区表兼容：处理分区字段 fieldId 不同位置导致的 Iceberg Partition Spec 兼容性问题
- 元数据生命周期管理：支持快照过期、元数据文件清理和版本保留配置

1.3 源码文件一览

文件名	类型	行数	职责描述
IcebergRestMetadataCommitter.java	主源码	473	REST Catalog 元数据提交器核心实现
IcebergRESTMetadataCommitterFactory.java	主源码	35	SPI 工厂类，创建 CommitterFactory 实例
IcebergRestMetadataCommitterTest.java	测试	854	REST 元数据提交器单元测试
IcebergMetadataTest.java	测试	589	Iceberg 元数据读写兼容性测试
IcebergDVCompatibilityTest.java	测试	315	Deletion Vector 兼容性测试
FlinkIcebergITCase.java	测试	129	Flink + Iceberg Hadoop Catalog 集成测试
IcebergRestMetadataCommitterITCase.java	测试	129	Flink + REST Catalog 集成测试

1.4 外部依赖关系

依赖模块/库	Scope	说明
paimon-core	provided	核心模块，提供 IcebergMetadataCommitter 接口、IcebergOptions、IcebergCommitCallback 等
paimon-bundle	provided	Paimon 打包模块
paimon-hive-catalog	provided	Hive Catalog 支持
iceberg-core 1.8.1	provided	Iceberg 核心库，提供 RESTCatalog、TableMetadata 等
hadoop-common	provided	Hadoop 公共库
paimon-flink-common	test	Flink 集成测试支持
jetty-server/servlet 11.0.24	test	REST 服务器测试依赖

第2章 核心架构设计

2.1 整体架构图

paimon-iceberg 模块的架构遵循分层设计原则，自上而下分为：SPI 工厂层、提交器实现层、REST Catalog 交互层。该模块作为 paimon-core 中 Iceberg 兼容框架的可插拔扩展，通过标准的 Factory SPI 机制集成。

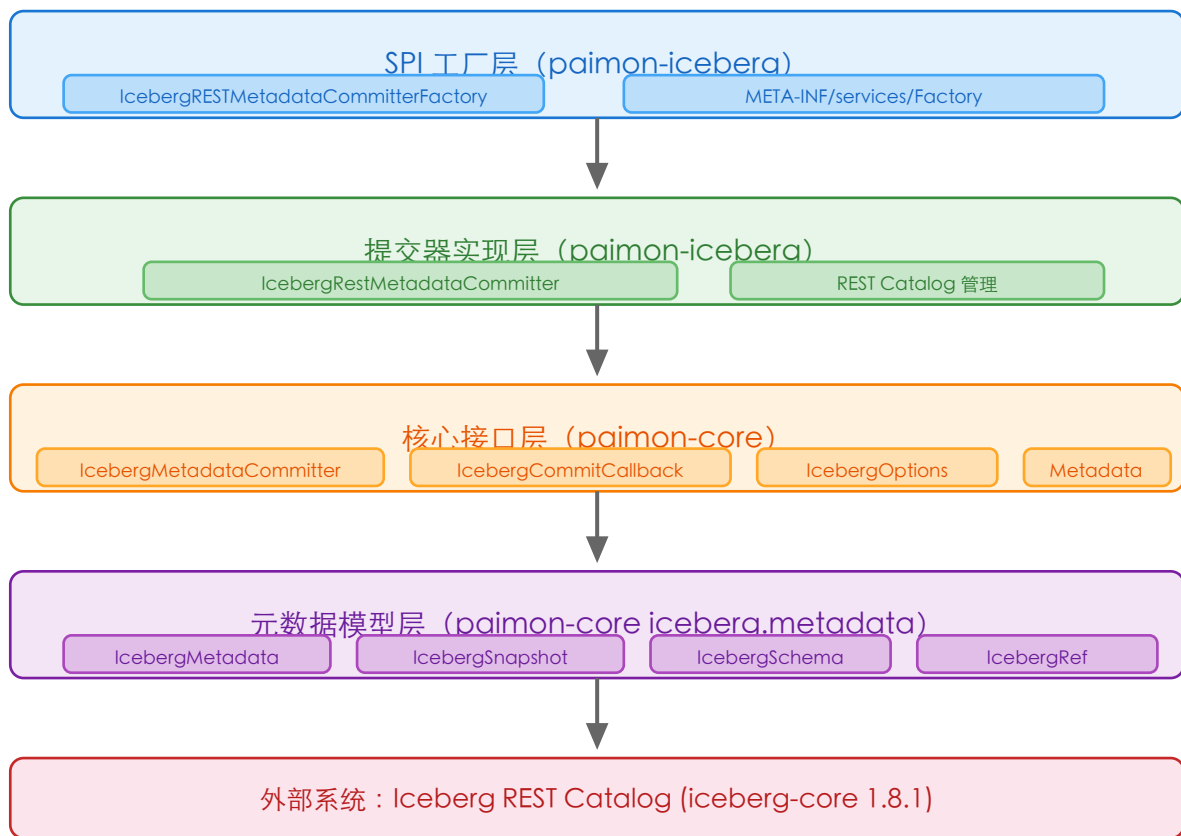


图2-1 paimon-iceberg 模块整体分层架构图

2.2 类关系图

paimon-iceberg 模块中的核心类关系如下所示。IcebergRESTMetadataCommitterFactory 实现 IcebergMetadataCommitterFactory 接口（继承自 Factory），负责通过 SPI 机制创建 IcebergRestMetadataCommitter 实例。IcebergRestMetadataCommitter 实现 IcebergMetadataCommitter 接口，内部持有 Iceberg RESTCatalog 实例和 IcebergOptions 配置。

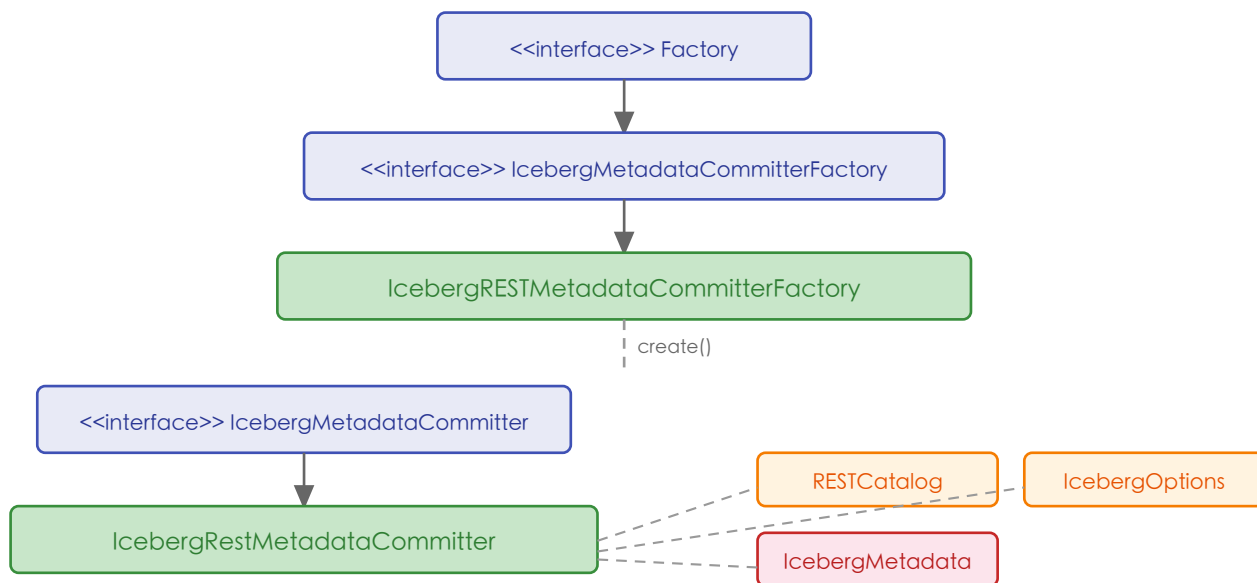


图2-2 核心类关系图

2.3 SPI 服务注册机制

paimon-iceberg 模块通过 META-INF/services/org.apache.paimon.factories.Factory 文件注册 IcebergRESTMetadataCommitterFactory 类。在 paimon-core 的 IcebergCommitCallback 构造函数中，通过 FactoryUtil.discoverFactory() 方法加载该工厂类，并调用 create(FileStoreTable) 方法创建提交器实例。

```
# META-INF/services/org.apache.paimon.factories.Factory
org.apache.paimon.iceberg.IcebergRESTMetadataCommitterFactory
```

第3章 核心类设计模式详解

3.1 IcebergRESTMetadataCommitterFactory — 工厂方法模式 (Factory Method)

`IcebergRESTMetadataCommitterFactory` 实现了 `IcebergMetadataCommitterFactory` 接口（继承自 `Paimon` 的 `Factory` 接口），是典型的工厂方法模式应用。该工厂通过 `Java SPI` 机制被自动发现，负责创建 `IcebergRestMetadataCommitter` 的具体实例。

模式说明：`Factory` 接口定义了 `identifier()` 和 `create()` 两个核心方法。`IcebergRESTMetadataCommitterFactory` 的 `identifier()` 返回 `"rest-catalog"` 字符串，与 `IcebergOptions.StorageType.REST_CATALOG` 枚举值匹配。`IcebergCommitCallback` 通过 `SPI` 发现所有 `Factory` 实现，根据配置选择正确的工厂并调用 `create(FileStoreTable)` 创建提交器。

关键方法：

- `identifier()` — 返回 `IcebergOptions.StorageType.REST_CATALOG.toString()`，即 `"rest-catalog"`
- `create(FileStoreTable table)` — 创建并返回 `IcebergRestMetadataCommitter` 实例

设计意图：通过 `SPI + Factory Method` 模式实现了存储类型的可插拔扩展。用户只需配置 `metadata.iceberg.storage=rest-catalog`，系统即可自动加载对应的提交器实现，无需修改核心代码。其他存储类型（如 `hive-catalog`、`hadoop-catalog`）可通过相同机制在各自模块中注册工厂。

3.2 IcebergRestMetadataCommitter — 策略模式 (Strategy)

`IcebergRestMetadataCommitter` 实现了 `IcebergMetadataCommitter` 接口，是策略模式的具体策略类。`IcebergCommitCallback` 持有 `IcebergMetadataCommitter` 接口引用，根据不同的 `identifier()` 调用不同的 `commitMetadata()` 方法。对于 `rest` 标识符，调用基于 `IcebergMetadata` 对象的提交方法；对于 `hive` 标识符，调用基于 `Path` 的提交方法。

策略选择逻辑（在 `IcebergCommitCallback` 中）：

```
switch (metadataCommitter.identifier()) {
    case "hive":
        metadataCommitter.commitMetadata(metadataPath, baseMetadataPath);
        break;
    case "rest":
        metadataCommitter.commitMetadata(metadata, baseMetadata);
        break;
}
```

设计意图：将元数据提交策略从 `IcebergCommitCallback` 中解耦出来，使得核心提交回调无需关心具体的 `Catalog` 类型，只需通过统一接口调用即可。新增 `Catalog` 类型时，只需新增对应的 `Strategy` 实现和 `Factory`，无需修改

IcebergCommitCallback。

3.3 IcebergRestMetadataCommitter — 适配器模式 (Adapter)

IcebergRestMetadataCommitter 本质上是一个适配器，将 Paimon 的 IcebergMetadata 模型适配为 Iceberg 原生的 TableMetadata 格式，并通过 Iceberg RESTCatalog API 进行提交。核心适配体现在两个方面：

- Schema ID 偏移适配：adjustMetadataForRest() 方法将 Paimon 中从 0 开始的 schemaId 调整为从 1 开始，以兼容 Iceberg REST Catalog 创建空 Schema 后占用 id-0 的情况
- TableMetadata 转换：通过 TableMetadataParser.fromJson(newIcebergMetadata.toJson()) 将 Paimon 的 IcebergMetadata JSON 解析为 Iceberg 原生的 TableMetadata 对象
- 分区规范适配：createTable() 方法根据分区字段 fieldId 是否为 0 采用不同的建表策略，避免 Iceberg 的分区演化问题

3.4 commitMetadataImpl — 模板方法模式 (Template Method)

commitMetadataImpl()

方法体现了模板方法模式的思想，定义了元数据提交的标准流程骨架：调整元数据

检查/创建数据库

→

检查/创建表

→

构建更新

→

提交更新。其中具体的更新构建逻辑根据是否有正确的 base metadata 分为两个分支方法：

- updatesForCorrectBase() — 基于正确 base 的增量更新逻辑
- updatesForIncorrectBase() — base 不一致时的全量重建逻辑

3.5 IcebergCommitCallback — 回调模式 (Callback) + 观察者模式 (Observer)

IcebergCommitCallback 同时实现了 CommitCallback 和 TagCallback 接口，是回调模式和观察者模式的结合。作为 Paimon 提交流程的观察者，它在快照提交成功后被回调执行 Iceberg 元数据同步工作。该类是整个 Iceberg 兼容层的中枢，负责：

- 收集文件变更（新增/删除）并转换为 Iceberg manifest 条目
- 构建和写入 Iceberg metadata JSON 文件
- 管理 manifest 压缩和快照过期
- 通过 SPI 委托给具体的 MetadataCommitter（如本模块的 REST 实现）进行最终提交
- 处理 Tag 的创建和删除（notifyCreation/notifyDeletion）

第4章 核心流程图

4.1 元数据提交主流程

当 Paimon 完成一次快照提交后，`IcebergCommitCallback.call()` 方法被回调，进而调用 `IcebergRestMetadataCommitter.commitMetadata()` 将 Iceberg 元数据同步到 REST Catalog。

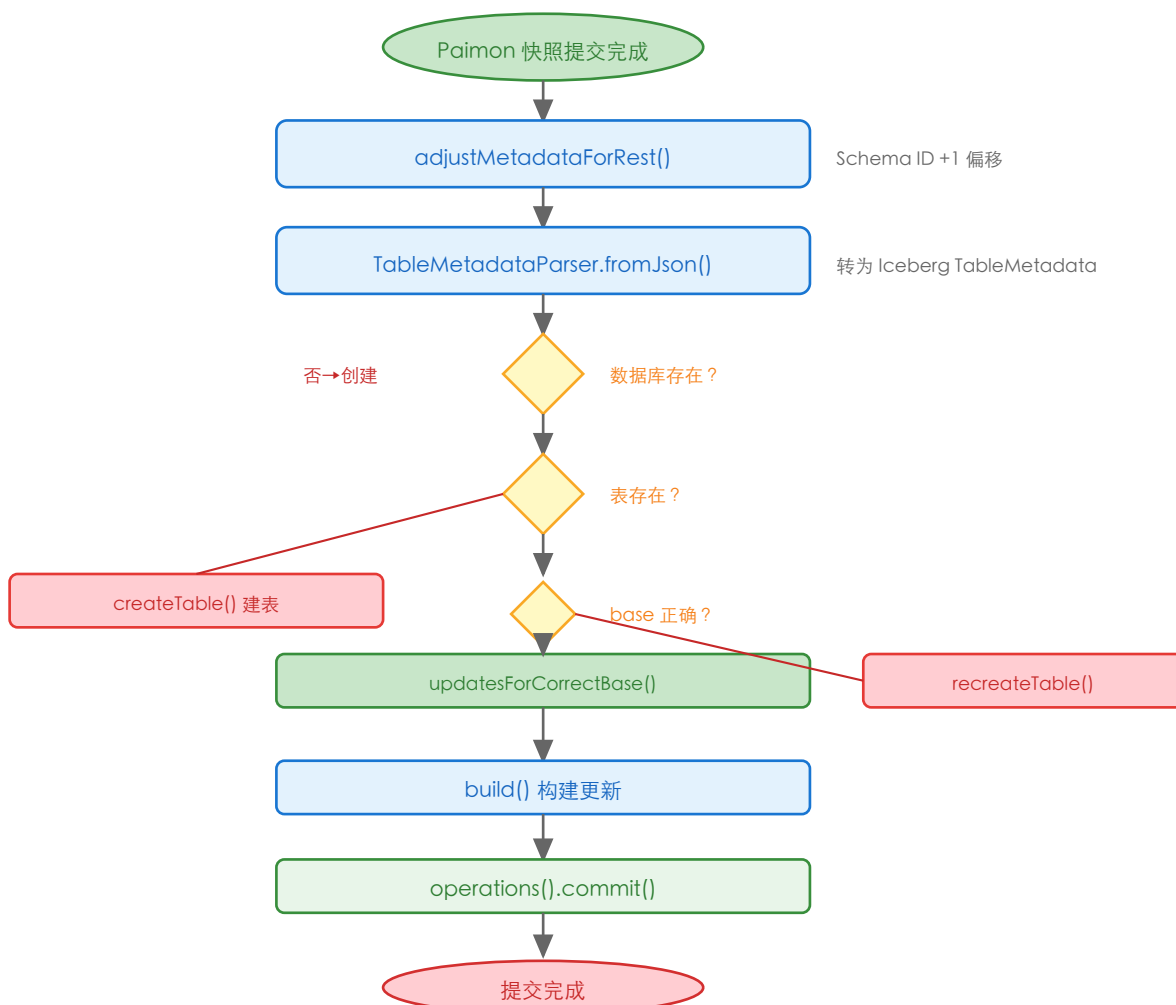


图4-1 REST Catalog 元数据提交主流程

4.2 表创建策略流程

`createTable()` 方法根据分区字段的 `fieldId` 位置采用不同的建表策略，以避免 Iceberg 的分区演化问题。Paimon 的 `fieldId` 从 0 开始，而 Iceberg 从 1 开始，直接转换会导致字段错位。

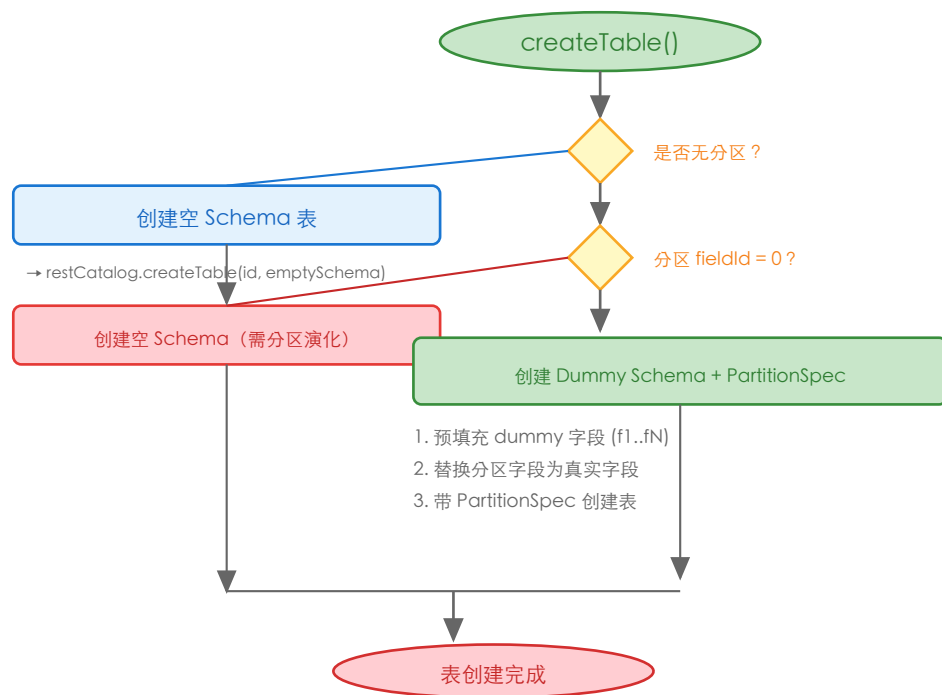


图4-2 表创建策略流程图

第5章 核心时序图

5.1 正常提交时序图（有 base metadata）

以下时序图展示了在 Paimon 快照提交后，IcebergCommitCallback 回调触发 REST Catalog 元数据同步的完整交互流程。

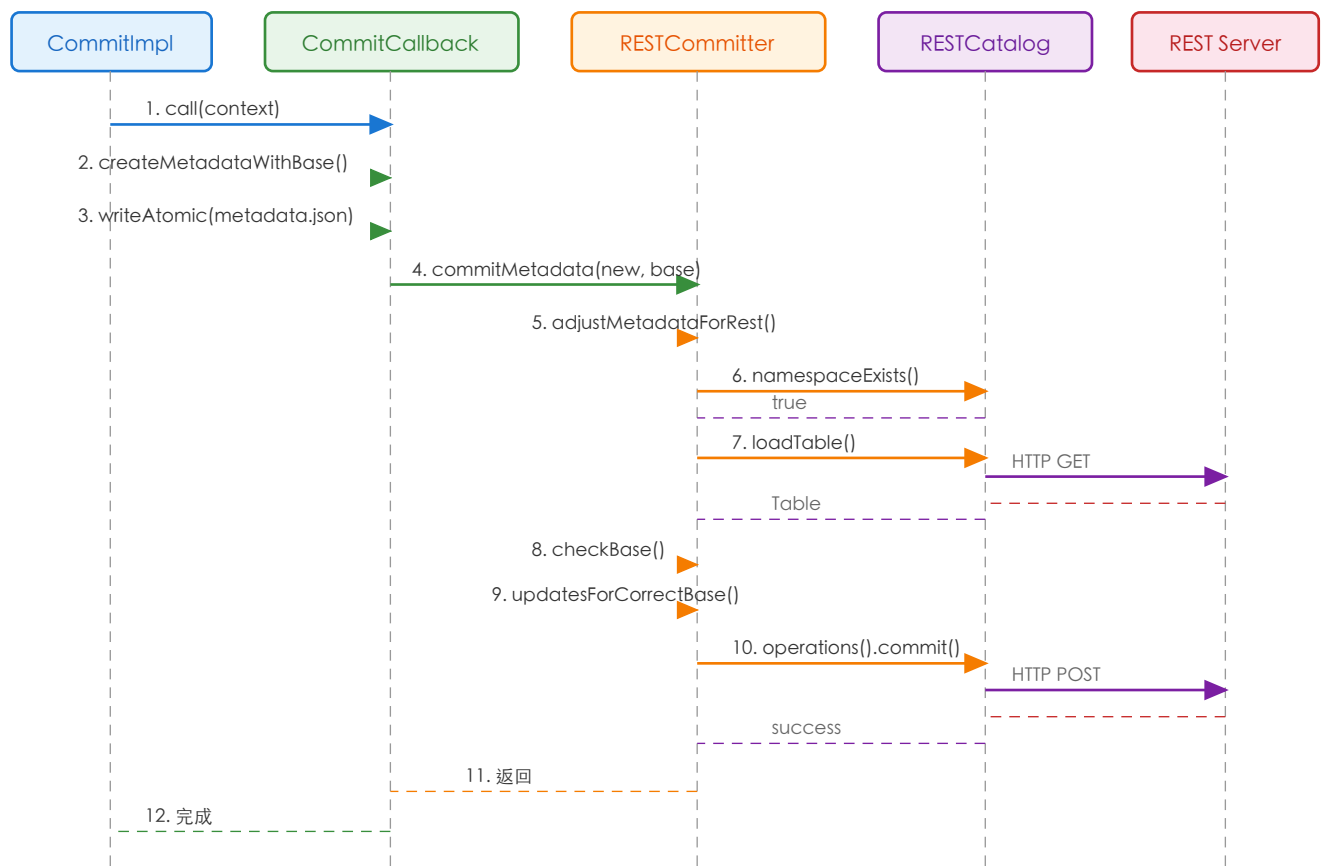


图5-1 正常提交时序图（有 base metadata）

5.2 异常恢复时序图（base 不一致）

当 Iceberg REST Catalog 中的当前快照与 Paimon 预期的 base 不一致时（例如中间有快照在 Iceberg 兼容未启用时提交），系统会执行 recreateTable() 进行全量重建。

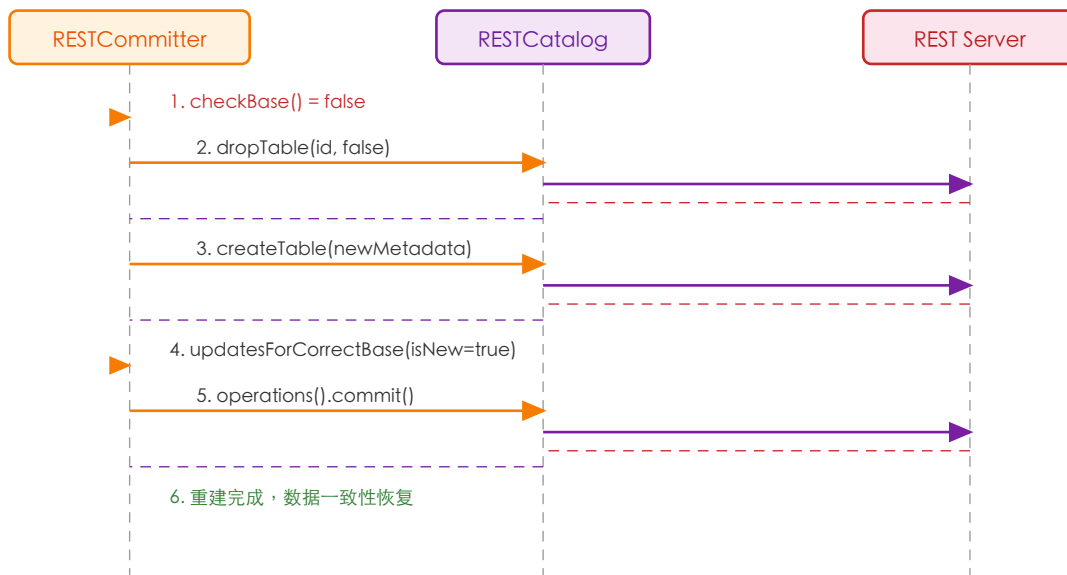


图5-2 base 不一致时的表重建时序图

第6章 关键场景调用链

6.1 场景一：首次提交（无 base metadata）

触发条件：Paimon 表首次启用 Iceberg REST Catalog 兼容，或之前的 Iceberg 元数据不存在。

调用链路：

步骤	类.方法	说明
1	IcebergCommitCallback.call(context)	Paimon 快照提交后回调入口
2	IcebergCommitCallback.createMetadata(snaps hot, ...)	判断是否存在 base metadata
3	IcebergCommitCallback.createMetadataWitho utBase(snapshotId)	全量创建 Iceberg 元数据（无 base）
4	SnapshotReader.read() → DataSplit.convertToRawFiles()	读取所有数据文件并转换为 RawFile
5	IcebergManifestFile.rollingWrite(entries, snapshotId)	写入 Iceberg manifest 文件
6	IcebergManifestList.writeWithoutRolling(metas)	写入 manifest list 文件
7	IcebergMetadata.toJson() → fileIO.tryWriteAtomic()	构建并写入 metadata JSON 文件
8	IcebergRestMetadataCommitter.commitMetad ata(metadata, null)	将元数据提交到 REST Catalog
9	adjustMetadataForRest(metadata)	调整 Schema ID 偏移（+1）
10	restCatalog.createNamespace() / createTable()	创建数据库和表
11	updatesForCorrectBase(base, newMetadata, isNew=true)	构建初始更新集
12	operations().commit(current, updated)	通过 REST API 提交到远端 Catalog

6.2 场景二：增量提交（有正确 base）

触发条件：Paimon 表已有 Iceberg 元数据，且 REST Catalog 中的当前快照 ID 与预期 base 一致。

调用链路：

步骤	类.方法	说明
1	IcebergCommitCallback.createMetadataWithBase(...)	基于已有 base metadata 增量创建
2	collectFileChanges(deltaFiles, removed, added)	收集本次快照的文件增删变更
3	createNewlyAddedManifestFileMetas()	为新增文件创建 manifest 条目
4	compactMetadataIfNeeded()	按需压缩 manifest 文件
5	IcebergRestMetadataCommitter.commitMetadata(new, base)	提交到 REST Catalog
6	checkBase(current, new, base)	验证 snapshotId 连续性
7	updatesForCorrectBase(base, new, false)	添加新 schema/snapshot，移除过期 snapshot
8	operations().commit(current, updated)	REST API 提交更新

6.3 场景三：base 不一致恢复

触发条件：REST Catalog 中的当前快照 ID 与新元数据预期的 base 不一致（中间有未同步的 Paimon 快照）。

调用链路：

步骤	类.方法	说明
1	checkBase() 返回 false	检测到 snapshotId 不连续
2	updatesForIncorrectBase(newMetadata)	进入不一致恢复分支
3	recreateTable(newMetadata)	重建 Iceberg 表
4	dropTable(icebergTableIdentifier, false)	删除旧表（不清除数据文件）
5	createTable(newMetadata)	按新元数据重新创建表
6	updatesForCorrectBase(base, new, isNew=true)	基于新空表构建全量更新
7	operations().commit(current, updated)	REST API 提交重建后的元数据

6.4 场景四：Schema 变更同步

触发条件：Paimon 表发生 Schema 变更（如新增列），需同步到 Iceberg REST Catalog。

调用链路：

步骤	类.方法	说明
1	SchemaManager.commitChanges(addColumn(...))	Paimon 端 Schema 变更
2	IcebergCommitCallback.createMetadataWithBase()	在下次提交时检测 schema 变更
3	SchemaCache.getLatestSchemaId()	获取最新 schema ID
4	IcebergSchema.create(tableSchema)	将 Paimon schema 转为 Iceberg schema
5	updatesForCorrectBase() → addAndSetCurrentSchema()	在 REST 端添加新 schema 并设为当前
6	update.addSchema(schema) / setCurrentSchema(id)	通过 Iceberg API 更新 schema
7	update.setProperties(properties)	同步元数据清理策略配置

6.5 场景五：Flink 端到端集成

触发条件：用户通过 Flink SQL 向 Paimon 表写入数据，配置了 metadata.iceberg.storage=rest-catalog。

调用链路：

步骤	类.方法	说明
1	Flink INSERT INTO paimon.default.T VALUES (...)	Flink SQL 写入 Paimon 表
2	TableWriteImpl.write() → commit()	Paimon 写入器写入并提交
3	IcebergCommitCallback.call()	快照提交后回调
4	IcebergRestMetadataCommitter.commitMetadata()	同步到 REST Catalog
5	Flink SELECT FROM iceberg.default.T	通过 Iceberg Flink Connector 查询
6	RESTCatalog.loadTable() → IcebergGenerics.read()	Iceberg 读取 Paimon 的原始数据文件

第7章 配置与扩展

7.1 核心配置属性

配置项	默认值	说明
metadata.iceberg.storage	disabled	Iceberg 存储类型：disabled/table-location/hadoop-catalog/hive-catalog/rest-catalog
metadata.iceberg.rest.uri	无	REST Catalog 服务 URI
metadata.iceberg.rest.warehouse	无	REST Catalog warehouse 位置
metadata.iceberg.rest.clients	无	REST Catalog 客户端连接池大小
metadata.iceberg.format-version	2	Iceberg 表格式版本，2 或 3（v3 支持 Deletion Vector）
metadata.iceberg.database	无	Iceberg 数据库别名，默认使用 Paimon 的数据库名
metadata.iceberg.table	无	Iceberg 表别名，默认使用 Paimon 的表名
metadata.iceberg.delete-after-commit.enabled	true	每次提交后是否删除旧的 metadata 文件
metadata.iceberg.previous-versions-max	0	保留的旧 metadata 文件最大数量（REST 至少保留 1）
metadata.iceberg.compaction.min-file-num	10	触发 manifest 压缩的最小文件数
metadata.iceberg.compaction.max-file-num	50	强制触发 manifest 压缩的文件数上限
metadata.iceberg.manifest-compression	snappy	Iceberg manifest 文件压缩编解码器
metadata.iceberg.storage-location	无	元数据存储位置：table-location/catalog-location

7.2 REST Catalog 配置

所有以 metadata.iceberg.rest. 为前缀的配置项都会被传递给 Iceberg RESTCatalog。例如 metadata.iceberg.rest.uri 会被转换为 Iceberg 的 uri 配置。配置提取逻辑在 IcebergOptions.icebergRestConfig() 方法中实现。

7.3 扩展点

paimon-iceberg 模块的主要扩展方式：

- 新增 Catalog 类型：实现 IcebergMetadataCommitterFactory 和 IcebergMetadataCommitter 接口，通过 META-INF/services 注册即可支持新的 Catalog（如 AWS Glue Catalog）
- 自定义数据库/表名映射：通过 metadata.iceberg.database 和 metadata.iceberg.table 配置项指定别名
- Deletion Vector 支持：设置 metadata.iceberg.format-version=3 并启用 deletion-vectors.bitmap64=true

第8章 设计亮点与总结

8.1 设计亮点

- SPI 可插拔架构：通过 Java SPI + Factory Method 模式实现了 Catalog 类型的零侵入式扩展。新增存储类型只需添加模块和 SPI 注册，无需修改 paimon-core 任何代码。
- Schema ID 偏移自适应：adjustMetadataForRest() 方法优雅地处理了 Paimon (ID从0开始) 与 Iceberg (ID从1开始) 之间的不兼容问题，通过统一偏移 +1 确保两端元数据一致。
- 分区字段兼容策略：createTable() 中根据分区 fieldId 位置采用两种不同策略（空 Schema vs Dummy Schema），巧妙地避免了 Iceberg 分区演化导致的查询引擎兼容性问题。
- 自动恢复机制：当检测到 base metadata 不一致时，系统自动执行 recreateTable() 进行全量重建，确保在中间有未同步快照的情况下也能恢复一致性。
- 增量更新优化：updatesForCorrectBase() 方法只同步增量变更（新 schema、新 snapshot、过期 snapshot 清理），避免每次都全量重建，大幅减少 REST API 调用开销。
- Deletion Vector 支持：通过 format-version=3 支持 Iceberg V3 规范的 Deletion Vector，实现了 Paimon DV 到 Iceberg puffin 格式的透明转换。

8.2 设计限制

- Path 方式提交未实现：commitMetadata(Path, Path) 方法抛出 UnsupportedOperationException，REST Catalog 只支持基于 IcebergMetadata 对象的提交方式。
- base 检查简单：checkBase() 仅通过 snapshotId 的连续性（current == new - 1）判断，不检查完整的元数据一致性，在极端并发场景下可能有误判。
- 模块源码精简：主源码仅 2 个文件（508行），大量核心逻辑依赖 paimon-core 中的 IcebergCommitCallback（1265行），模块本身主要承担适配和桥接角色。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	分层清晰，职责单一，SPI 可插拔设计优秀
设计模式运用	★★★★★	Factory Method + Strategy + Adapter + Template Method 多模式协同
可扩展性	★★★★★	通过 SPI 机制支持零侵入式扩展新 Catalog 类型
异常处理	★★★★☆	关键操作有 try-catch，但部分异常信息可更详细
测试覆盖度	★★★★★	5 个测试类覆盖单元测试、集成测试、DV 兼容性测试
文档完整性	★★★☆☆	类级 Javadoc 较少，内部方法多为简短注释

8.4 概念映射表

以下表格展示了 Paimon 概念与 Iceberg 概念之间的对应关系：

Paimon 概念	Iceberg 概念	说明
Snapshot	Snapshot	Paimon snapshotId 直接映射为 Iceberg snapshotId
TableSchema	Schema	通过 IcebergSchema.create() 转换，field ID 从 0 偏移到 1
DataFileMeta / RawFile	DataFile	通过 IcebergDataFileMeta.create() 转换
IcebergMetadata (JSON)	TableMetadata	通过 TableMetadataParser.fromJson() 解析
PartitionKeys	PartitionSpec	通过 IcebergPartitionField 映射
Manifest	ManifestFile	Paimon 生成 Iceberg 兼容的 manifest avro 文件
Tag	Ref (branch)	Paimon tag 映射为 Iceberg snapshot ref
DeletionVector (bitmap64)	DeletionFile (puffin)	仅 format-version=3 时支持